OpenZeppelin | security

Miden Multisig Cryptography Audit

MIDEN

March 17, 2026

Table of Contents

Table of Contents 2

Summary 4

Scope 5

System Overview 7
- Summary 7
- Components 7

Security Model and Trust Assumptions 8
- Privileged Roles 9

High Severity 10
- H-01 Participants Sign Unvalidated Transactions 10
- H-02 Multisig Client Cannot Support Offline Use Case 11

Medium Severity 12
- M-01 PSM Can Permanently Corrupt Client State 12
- M-02 Defensive Programming 13
- M-03 PSM Endpoint Is Not Verified 14
- M-04 Inconsistent Proposal Retrieval 15
- M-05 Rust Client Ignores Procedure Threshold Overrides 16
- M-06 PSM Authentication Ignores Request Details 16
- M-07 Inconsistent and Redundant Proposal Fields 17
- M-08 P2ID Note Serial Number Derivation Differs Between TS and Rust Clients 18
- M-09 Non-Canonical signerId Strings Can Collide After Normalization 18
- M-10 Signer Updates Can Make Stored Per-Procedure Thresholds Permanently Unreachable 19

Low Severity 21
- L-01 Secret Key Material Susceptible to Exposure 21
- L-02 Typescript Client Accepts Stale State 22
- L-03 Unreachable Code 22
- L-04 Redundant PSM Endpoint Field 22
- L-05 TypeScript Client Multisig load() Creates Null-Account Instances 23
- L-06 Code Simplification 23
- L-07 Documentation Suggestions 25
- L-08 Users Can Sign Unrelated Proposal 26
- L-09 Wraparound Risk for Untrusted Inputs in Hex-to-Word Parsing 26

Notes & Additional Information 27
- N-01 Ambiguous Signer Map Selection When Both OZ and Miden Slots Are Present 27

N-02 Incomplete Proposal Description 27
N-03 Inconsistent Codebase 28
N-04 Unused Code 28

Client Reported 29
CR-01 PSM Acknowledgement Never Expires 29

Conclusion 30

Appendix 31
Issue Classification 31

Summary

Type	Cryptography	Total Issues	26 (24 resolved)
Timeline	From 2026-02-02 To 2026-02-13	Critical Severity Issues	0 (0 resolved)
Languages	Rust TypeScript	High Severity Issues	2 (2 resolved)
		Medium Severity Issues	10 (9 resolved)
		Low Severity Issues	9 (9 resolved)
		Notes & Additional Information	4 (4 resolved)

Scope

OpenZeppelin audited the multisig wallet feature of the OpenZeppelin/private-state-manager repository at commit 7aa4461 (branch name `upgrade-to-v0.13`).

In scope were the following files:

```
crates
└── miden-multisig-client
    └── src
        ├── account.rs
        ├── builder.rs
        ├── client
        │   ├── account.rs
        │   ├── helpers.rs
        │   ├── io.rs
        │   ├── mod.rs
        │   ├── notes.rs
        │   ├── offline.rs
        │   └── proposals.rs
        ├── config.rs
        ├── error.rs
        ├── execution.rs
        ├── export.rs
        ├── keystore.rs
        ├── lib.rs
        ├── payload.rs
        ├── procedures.rs
        ├── proposal.rs
        └── transaction
            ├── builder.rs
            ├── configuration.rs
            ├── consume.rs
            ├── mod.rs
            ├── payment.rs
            └── psm.rs

packages
└── miden-multisig-client
    ├── src
    │   ├── account
    │   │   ├── builder.ts
    │   │   ├── index.ts
    │   │   ├── masm.ts
    │   │   └── storage.ts
    │   ├── client.test.ts
    │   ├── client.ts
    │   ├── index.ts
    │   ├── inspector.test.ts
```

```
    │   ├── inspector.ts
    │   ├── multisig.test.ts
    │   ├── multisig.ts
    │   ├── multisig
    │   │   ├── helpers.test.ts
    │   │   └── helpers.ts
    │   ├── procedures.ts
    │   ├── signer.test.ts
    │   ├── signer.ts
    │   ├── transaction.test.ts
    │   ├── transaction.ts
    │   ├── transaction
    │   │   ├── consumeNotes.ts
    │   │   ├── index.ts
    │   │   ├── options.ts
    │   │   ├── p2id.ts
    │   │   ├── summary.ts
    │   │   ├── updatePsm.ts
    │   │   └── updateSigners.ts
    │   ├── types.ts
    │   ├── types
    │   │   └── proposal.ts
    │   └── utils
    │       ├── encoding.test.ts
    │       ├── encoding.ts
    │       ├── index.ts
    │       ├── random.ts
    │       ├── signature.test.ts
    │       ├── signature.ts
    │       ├── word.test.ts
    │       └── word.ts
    └── vitest.config.ts
```

System Overview

Summary

This audit covers the Multisig Private State Manager (PSM) client implementations in Rust (`crates/miden-multisig-client`) and TypeScript (`packages/miden-multisig-client`). Both clients provide a multi-signature workflow on top of Miden's account model: they generate transaction summaries locally, collect cosigner signatures, and coordinate with a PSM server to exchange proposals, signatures, and acknowledgements before finalizing execution on the Miden network. The Rust client targets native/CLI and server-side usage, while the TypeScript client targets web applications and browser-based workflows. The two codebases implement the same core protocol behaviors and are expected to be functionally aligned.

The PSM acts as an off-chain coordinator and storage layer for account snapshots, deltas, and multi-party proposals. It is not a trustless component: it is an availability and data-integrity dependency, and it may be self-hosted or managed by a third party. The clients therefore enforce local validation, threshold logic, and signature construction while relying on the PSM for proposal storage and multi-party coordination. On-chain validation ultimately enforces that only valid signatures from current signers can authorize state transitions.

Components

The audited clients can be viewed as a pipeline with four main stages:

1. **Account and Configuration Handling**
    - Deserialize and inspect Miden accounts.
    - Extract multisig configuration (signer commitments, threshold, PSM commitment/endpoint, procedure overrides).
    - Initialize client state for signing and proposal tracking.
2. **Proposal Creation and Signing**
    - Build transaction requests and compute transaction summary commitments.
    - Create proposal payloads (summary + metadata) and submit them to the PSM.
    - Allow cosigners to sign proposals and submit signatures to the PSM.

3. **Proposal Retrieval and Execution**
    - Fetch proposals/delta proposals from the PSM.
    - Determine readiness based on signature collection and configured thresholds.
    - Construct execution advice from cosigner signatures and (when required) PSM acknowledgements.
    - Submit finalized transactions to the Miden network.
4. **Sync and State Management**
    - Synchronize local state with PSM snapshots and/or the Miden network.
    - Resolve commitment mismatches and track account nonces.
    - Export/import proposals for offline workflows (Rust and TS).

The Rust and TypeScript clients implement the same logical responsibilities but differ in API shape and runtime environment. Cross-implementation divergence and inconsistent validation paths are therefore a key review focus.

Security Model and Trust Assumptions

The multisig clients have been designed under the following assumptions:

- **PSM is semi-trusted for availability and coordination, not for authorization:** The PSM can store and relay proposals and signatures, but it does not possess cosigner private keys. It can be malicious or unavailable. Thus, the clients should not accept PSM-provided data without validation where feasible.
- **Authorization is enforced by cosigner signatures and on-chain checks:** Even if a malicious PSM injects or reorders data, the on-chain multisig logic should reject invalid signatures or transactions that do not meet the threshold of current signers.
- **Clients are responsible for local validation and correct construction:** The clients must validate signer sets, thresholds, metadata, and proposal integrity before execution. Mistakes here can lead to DoS, confusing UX, or unintended transaction construction, even if on-chain verification prevents outright bypass.

- **Key material is assumed to be managed safely by the host environment:** The audited code includes key usage and signing logic but generally relies on the caller to provide secure key storage and lifecycle management.
- **Cross-implementation consistency matters:** Since the Rust and TypeScript clients are expected to follow the same protocol rules, differences in validation, parsing, or signature handling can introduce exploitable mismatches or operational failures.

Privileged Roles

Throughout the codebase, the following privileged roles were identified

- **Multisig cosigners** (holders of signing keys; authorize proposals and execution)
- **Proposal creator/proposer** (a cosigner; initiates transactions)
- **PSM operator** (controls storage/coordination service and ACK signing key)
- **PSM acknowledgement key holder** (signs `ack_sig`; often the same as the operator)
- **Miden network validators/sequencer** (enforces on-chain signature checks)
- **Client operator** (entity running the Rust/TS client with access to local keys/state)

High Severity

H-01 Participants Sign Unvalidated Transactions

The multisig client does not have any mechanism to validate the transaction being signed. Thus, users are forced to trust the PSM. In particular, when listing the proposals, the client extracts the transaction summary to be signed and the metadata that describes the transaction. However, the client does not validate that these values are consistent with each other. If the PSM provided the metadata describing a different operation, the client (and therefore the user) would not recognize the discrepancy. Later, when signing the proposal, the user would specify the proposal ID to be signed while believing it corresponds to a different operation.

Specifically in the case when a proposal is imported offline, there is a missing binding between the proposal ID and the commitment to the transaction summary: `proposal.id == commitment(tx_summary)`. In this scenario, the client does not validate that the serialized `id` matches the commitment of the serialized `tx_summary`. It only checks that the exported account ID (when available) matches the ID of the account and `ExportedProposal::to_proposal()` copies `self.id` verbatim into the reconstructed `Proposal`. Signing similarly derives the commitment from `tx_summary` and does not check `id` consistency. This allows a tampered artifact to present an expected `id` while binding signatures to a different `tx_summary`, enabling proposal-ID spoofing and increasing the risk of social engineering in offline coordination.

Consider ensuring that the metadata provided by the PSM (or an exported proposal) matches the transaction summary, stressing that in the exported case, the proposal ID should also be derived from the transaction summary. This likely involves reconstructing the transaction to retrieve the summary. Alternatively, instead of signing the transaction summary, consider signing (the hash of) a `Proposal` object. This would be simpler to coordinate but would require the on-chain account to be updated accordingly.

Update: *Resolved in pull request #137. The team stated:*

> *Created PR that prevents signing/executing tampered proposals by enforcing consistency checks in both Rust and TypeScript multisig clients.*
>
> *- Validate `proposal.id == commitment(tx_summary)`*

> *- Rebuild tx request from `metadata` and verify it matches the stored `tx_summary` commitment*
>
> *- Validate `tx_summary` for exported/imported proposals*

H-02 Multisig Client Cannot Support Offline Use Case

The multisig client has an "offline" mode for when the PSM is unavailable. However, it still syncs with the PSM when creating and executing proposals, and it still requires a PSM acknowledgement (enforced by the Miden account), which undermines the whole use case. Note that the typescript client also supports offline signing, but only provides an online execute function.

Consider bypassing the PSM sync in offline mode noting that since the PSM acknowledgement is required by the account, in practice, only "Switch PMS" proposals can have offline support. Therefore, consider restricting (and simplifying) the offline sequence to only this proposal type. Alternatively, consider redesigning the account logic so that a PSM acknowledgement is not required if all possible signatures (n-of-n) are provided. This guarantees that all cosigners have a copy of the data.

Update: *Resolved in pull request #138. The team stated:*

> *Restrict Rust multisig offline flow to `SwitchPsm` only and remove misleading offline paths for other proposal types:*
>
> *- adds a network-only sync for offline operations (no PSM sync)*
>
> *- simplifies the offline logic, restricted only to PSM switch operations*
>
> *- centralizes transaction capability checks (`supports_offline_execution` / `requires_psm_ack`),*

Medium Severity

M-01 PSM Can Permanently Corrupt Client State

The PSM is a semi-trusted entity. It is expected to track the account's private state and reveal it to cosigners on request. However, the system includes safeguards to bypass or change the PSM if it becomes non-responsive or malicious. Thus, it should be ensured that this replacement operation is always possible.

Specifically, the Rust and TypeScript clients ingest account state from the PSM and treat it as authoritative without validating that the state is both authentic (matches on-chain commitment) and structurally valid for a multisig account (expected components, thresholds, signer map, PSM commitment layout, etc.). This creates risks such as a malicious or buggy PSM getting desynchronized or bricking clients by overwriting state, and malformed yet "on-chain-valid" state can still crash or mislead the client's multisig logic.

The client will routinely retrieve the new state from the PSM and overwrite its own local copy, as well as the copy in its associated Miden client. This means that a misbehaving PSM can corrupt the account data held by all cosigners. Since the Miden network cannot track private account data (which is why the PSM is needed in the first place), corrupting all privately held account data can make the account unusable.

Furthermore, the Rust client implementation deserializes and interprets account state data provided by the PSM. Similar functionality is implemented in the TypeScript client. However, in both cases, there is missing client-side validation of the integrity and correctness of the incoming data. Consequently, malformed or malicious account state data may lead to client-side crashes, incorrect interpretation of the multisig configuration, or an inconsistent local view of the account that could enable further attacks.

Consider validating the new PSM state against the on-chain account commitment before accepting it. In addition, consider exporting the account data (to a backup file) before overwriting it. This would simplify any necessary recovery option. Additionally, consider adding validation checks of the deserialized account state. For instance, ensure that it contains the expected multisig and PSM components, that it has a valid threshold and number of signers, validates the PSM commitment, etc.

Update: *Resolved in pull request #148 at commit 4f7d6a7. The team stated:*

> *Fix account commitment (state) validation before overriding local state. Updated `get_deltas()` to validate sync correctness using the local account computed commitment vs on-chain commitment.*

M-02 Defensive Programming

Several issues in this report are a consequence of minimal validation or high coupling across the whole system, which increases fragility. The codebase could benefit from an incremental cleanup with a view towards defensive programming. The intention would be to create well-defined boundaries between the components and heavily scrutinize messages that cross those boundaries. It would also involve failing early wherever possible.

Some instances where defensive programming can be used are listed below:

- The `import_proposal` function accepts an arbitrary `.json` file that has the correct account ID. It should validate the rest of the structure to ensure it has the required metadata fields and signatures. Similarly, the `importProposal` function should ensure that the nonce is defined.
- The `create_account_with_proc_thresholds` function does not perform any duplicate checks on the `signer_commitments` input. Similarly, the `validateMultisigConfig` function in the TypeScript client does not check for duplicates.
- When creating a new proposal, the proposal ID is copied from the PSM response. This should be validated to match the transaction summary.
- The `executeProposal` function in the TypeScript client builds advice from `proposal.signatures` without validating signer commitments against the current signer list. Consequently, the client may try to execute a proposal with a set of signatures that appears to meet the threshold due to the presence of non-cosigner signatures. This will ultimately fail during on-chain verification, where each signature is verified against the current signer public keys stored in the account, but it is unnecessarily fragile. For comparison, the Rust client filters the commitments of the cosigners' public keys in `collect_signature_advice` called in `execute_proposal` during proposal execution.
- In `get_psm_ack_signature`, the client fetches the PSM public key from the server and uses it directly for advice without cross-checking against the PSM commitment stored in the account. This may interrupt normal execution if the endpoint is wrong or malicious. Consider verifying that the PSM public key commitment obtained from the `ack` matches the one from the account storage (e.g., as returned from `psm_commitment`).

- The `signProposal` function passes no metadata to the `deltaToProposal` function, and then just overwrites the metadata that it computes. This implies that it will throw an error if it cannot parse the delta metadata, and it will use the delta metadata's proposal type to derive the status, even though it is irrelevant and may be inconsistent with the actual metadata recorded with the proposal.

Consider addressing the above-listed instances of defensive programming and reviewing the entire codebase with a view towards defensive programming.

Update: *Resolved in pull request #148 at commit 0b20337. The team stated:*

> *- hardened proposal import in rust and typescript with full validation, not just checked by* `account_id`
>
> *- added validations for proposal identity by verifying that proposal commitments match the underlying* `tx_summary`
>
> *- reject duplicate signer commitments during multisig creation in both rust and typescript*
>
> *- reject malformed, duplicate, and non-valid signatures at proposal fetch/import boundaries*
>
> *- verify the PSM acknowledgment pubkey commitment against the accounts stored PSM commitment before using ack*
>
> *- removed fragile typescript proposal re-parsing during signing by reusing validated local metadata instead of trusting server metadata again.*
>
> *- minor refactors*

M-03 PSM Endpoint Is Not Verified

The `SwitchPSM` transaction includes a new commitment and endpoint, but only the commitment is relevant to the transaction. This means that the endpoint cannot be validated from any provided metadata.

Instead of simply trusting the provided endpoint, consider requiring the new PSM to sign a message that can be checked against the new commitment. This could potentially be moved to the PSM client itself so that all messages from the PSM are authenticated with the PSM commitment.

Update: *Resolved in pull request #148 at commit f50fa53. The team stated:*

> *`switch_psm` now uses endpoint vs commitment verification in both Rust and TypeScript:*
>
> *- on proposal create/execute, the client calls the new endpoint's `/pubkey` and requires it to match the proposed new PSM commitment, otherwise it aborts.*
>
> *- After successful on-chain execution, the client switches to that endpoint and registers synced local account state on the new PSM*
>
> *- For state safety checks, on-chain validation is enforced only when a real on-chain commitment exists, missing/zero commitment is treated as "not yet on-chain"*

M-04 Inconsistent Proposal Retrieval

When executing a proposal, the rust client requests the raw delta values from the PSM, and then lists the parsed proposals, which also implicitly requests the same delta values in a separate call to the PSM. This involves unnecessary duplication, but we expect the PSM to return the same values in both cases (unless it acts maliciously).

However, the parsed proposals are filtered by proposal ID and the raw delta values are filtered by nonce. Since the proposal nonce is derived from the account nonce at proposal construction time, it is possible for multiple pending proposals to have the same proposal nonce. This means we may retrieve the signatures from a different proposal, causing the eventual transaction execution to fail.

Consider retrieving the proposals and signatures using a single PSM query, and filtering on the unique proposal ID.

Update: *Resolved in pull request #148 at commit bb17b50. The team stated:*

> _`execute_proposal` now uses a single PSM proposal fetch and match proposals strictly by `proposal_id` (not by nonce), then use the raw delta from that exact matched item. _
>
> *This removes the bug where two pending proposals with the same nonce could cause execution to pick signatures from the wrong proposal*

M-05 Rust Client Ignores Procedure Threshold Overrides

A `MultisigAccount` may be created with procedure threshold overrides, which change the threshold (number of signatures required) on a per-function basis. However, the Rust client always assumes that the default threshold is active. This means that proposals that should have lower thresholds may be incorrectly labeled as `Pending`, preventing the execution of valid transactions.

Consider incorporating the per-procedure threshold when determining the proposal status.

Update: *Resolved in pull request #148 at commit 1fa9f48. The team stated:*

> *Added support to threshold overrides in rust multisig client*

M-06 PSM Authentication Ignores Request Details

The `PsmClient` calls the `add_auth_metadata` function to authenticate with the PSM. However, the signature only covers the account ID and timestamp and does not include any details about the intended action. In principle, this implies that an attacker (with sufficient network access) could reuse that signature on an unrelated operation.

Consider including the request message as part of the data to be signed.

Update: *Resolved in pull request #148 at commit 25b8f4c. The team stated:*

> *PSM auth request bound instead of just account+time bound*
>
> *- We introduced a shared auth message format: `account_id` + `timestamp` + `request_payload_digest`.*
>
> *- `request_payload_digest` is computed from the request content:*
>
> *- gRPC: hash of protobuf bytes.*
>
> *- HTTP/TS: hash of canonicalized JSON.*
>
> *- Signers now use `signRequest()` (not just account/timestamp), and server verification checks the same full message.*

> *Net effect: a signature is valid only for one specific request payload, so cross-endpoint/action replay with the same timestamp is blocked.*

M-07 Inconsistent and Redundant Proposal Fields

The `ProposalStatus` enum may include the number of required and collected signatures, even though this is already tracked by the `ProposalMetadata` struct. Moreover, the number of collected signatures is implied by the list of signatures. This introduces the possibility of inconsistencies. For example, new proposals set the `signatures_collected` field in the status to zero, which matches the empty list of signers, regardless of whether the corresponding field in the metadata is zero.

Furthermore, the `signatures_required` variable is set to the number of signers (so we have an n-of-n multisig) instead of retrieving it from the metadata. In addition, when validating a transaction for execution, the cosigner signatures are independently retrieved from the PSM. Apart from this, the `has_signed` function will return false in the `Ready` or `Finalized` state, even if the specified signer was used to get to that state. This means a user can always sign a proposal that is not `Pending`.

Consider moving the `signers` field to the `ProposalMetadata` struct, removing the redundant `signatures_required` field, and removing both `signatures_collected` and `collected_signatures` fields (instead deriving this value from the length of the signers).

Update: *Resolved in pull request #148 at commit 7d1ae30 and at commit 87816ea. The team stated:*

> *- `ProposalStatus` was simplified to `Pending | Ready | Finalized` (no embedded signature fields).*
>
> *- Signer tracking moved to `ProposalMetadata.signers`. Redundant `collected_signatures` and status-level signature counters were removed.*
>
> *- Signature counts are now derived from `metadata.signers.len()` plus `metadata.required_signatures`.*

M-08 P2ID Note Serial Number Derivation Differs Between TS and Rust Clients

The TS SDK reimplements P2ID note construction in `buildP2idNote` and hard-codes the note recipient `serialNum` as `Rpo256.hashElements([...salt.toFelts(), 0])`. In contrast, the Rust SDK seeds an RNG from the same salt (`RpoRandomCoin::new(salt)`) and passes it to `miden_standards` in `build_p2id_transaction_request`, delegating serial number derivation to the standard `create_p2id_note` constructor.

Since the serial number is part of the note recipient, a mismatch changes the output note commitment and therefore changes the `TransactionSummary` commitment that cosigners sign. The execution path in `Multisig.executeProposal` derives `saltHex` from `txSummary.salt()` and then rebuilds the final request by calling `buildP2idTransactionRequest`. If the stored proposal `txSummary` had been created by the Rust construction, the rebuilt TS request produces a different commitment. Thus, signature advice is prepared for the wrong message and execution fails.

Consider aligning P2ID note construction across SDKs by deriving `serialNum` the same way as the Rust path (i.e., by delegating to the standard `create_p2id_note` construction using an RNG seeded from `salt`, instead of hard-coding `Rpo256.hashElements(salt || 0)`). Moreover, consider adding cross-SDK interoperability tests that assert identical output note commitments and `TransactionSummary` commitments for fixed inputs.

Update: *Resolved in pull request #148 at commit b0fc1a8 and at commit 1dd97c5. The team stated:*

> *Fixed typescript P2ID serial derivation by padding the salt hash input with four 0 felts matching rust implementation (`RpoRandomCoin`)*

M-09 Non-Canonical `signerId` Strings Can Collide After Normalization

The TypeScript multisig client aggregates signatures from multiple sources (existing local signatures and server-provided `cosignerSigs`) when building proposals. In `deltaToProposal`, the merge logic de-duplicates with a `Map` keyed by the raw `signerId` string. Proposal readiness is also based on raw counts, both in `executeProposal` (`proposal.signatures.length`) and when mapping server status in `deltaStatusToProposalStatus` (`status.cosignerSigs.length`). Therefore, during the counting stage (i.e., before execution), multiple signatures for the same signer in a given

proposal would be counted as distinct under different string formatting (e.g., `signerId = "0x1"` and `signerId = "0x000...001"`).

In contrast, during execution, `executeProposal` canonicalizes identities by converting `Word.fromHex(normalizeHexWord(cosignerSig.signerId))`, where `normalizeHexWord` lower-cases and left-pads the hex string to 32 bytes. Consequently, the same signer under different string formatting (e.g., `signerId = "0x1"` and `signerId = "0x000...001"`) collapses to the same normalized `Word` (signer commitment). Afterwards, this Word is used to derive the advice-map key via `buildSignatureAdviceEntry` which is then inserted into `AdviceMap` in a loop.

This inconsistency allows a proposal (including an offline-imported proposal via `importProposal`) to carry multiple distinct `signerId` encodings that normalize to the same signer commitment. The duplicates can prematurely satisfy the "ready" gate, and the normalization-colliding entries can also overwrite each other in the `AdviceMap`, replacing a valid signature with an invalid one and causing deterministic proving and execution failure.

Consider canonicalizing `signerId` at all ingestion points (server sync, offline import, and local signing) by applying `normalizeHexWord` and rejecting non-32-byte encodings, and then de-duplicating and computing readiness based on unique normalized commitments that belong to the expected signer set. Consider also detecting duplicate normalized commitments during `executeProposal` and treating advice-map key collisions as an error instead of silently overwriting.

_**Update:** Resolved in pull request #148 at commit 6c6c032. The team stated:

> *Implemented strict signer canonicalization and duplicate rejection across the tyescript multisig client.*
>
> *All signatures are now normalized to 32 bytes commitments and validated against the multisig signer set at sync, import, online/offline sign and execute flows.*

M-10 Signer Updates Can Make Stored Per-Procedure Thresholds Permanently Unreachable

The `auth_tx_rpo_falcon512_multisig` authentication procedure enforces a single `transaction_threshold` for the entire transaction. This value is computed by `compute_transaction_threshold` as the maximum per-procedure override found in `PROC_THRESHOLD_ROOTS_SLOT` among procedures called in the transaction.

However, while `update_signers_and_threshold` updates the threshold config and signer public keys, it does not read or update `PROC_THRESHOLD_ROOTS_SLOT`. As such, after a signer removal, previously valid overrides can become greater than the new `num_of_approvers`, and `auth_tx_rpo_falcon512_multisig` will still enforce the now-unreachable `transaction_threshold` (it does not bound the computed threshold by the current signer count). Therefore, `update_signers_and_threshold` updates `num_of_approvers` without enforcing that all stored per-procedure thresholds remain `<= num_of_approvers`. This can permanently brick transactions that call affected procedures, and can also brick governance if the `update_signers` procedure itself is configured with an unreachable override.

Inside `update_signers_and_threshold`, consider enforcing that each stored per-procedure threshold is `<= num_of_approvers` after the update (e.g., revert if any override exceeds the new signer count, or clamp/clear invalid overrides). Consider also providing a dedicated procedure to update or clear per-procedure overrides so that the configuration can be repaired without relying on off-chain tooling.

Update: *Acknowledged, will resolve. We acknowledge that the issue is addressed in pull request #148 at commit a5b9d71. As the issue is out-of-scope, the changes introduced in this commit have not been reviewed. The team stated:*

> *Fixed:*
>
> *- Added a new multisig proposal type, `update_procedure_threshold`, so a procedure threshold overrides can be updated.*
>
> *- Fixed the core audit issue in the MASM contract: `update_signers_and_threshold` now rejects signer updates that would leave any stored procedure override above the new signer count*
>
> *- Updated the rust and typescript multisig stack to support the new transaction type end to end*

Low Severity

L-01 Secret Key Material Susceptible to Exposure

The cosigner's secret key is stored as a struct field and can be obtained via `secret_key` and `clone_secret_key`. This makes accidental or malicious leakage much easier (core dumps, process compromise, unintended logging or passing across components). While memory zeroization is not a full solution - especially given the fact that the key is originally stored in memory - uncontrolled duplication is still a concrete risk. It also blocks future hardening (e.g., HSM keystore or out-of-process signer) because the API assumes raw key access.

The risk partially applies to the TypeScript client as well, which also stores the secret key in memory. However, in contrast to the Rust client, it does not support a cloning or getter functionality.

To address the noted risk, consider applying the following recommendations to the Rust client and, where applicable, to the TypeScript PSM client:

- Eliminate `clone_secret_key` and any direct key getters so that the key never leaves the key manager.
- Treat the key manager as a self-contained signer. Pass references to it (or send requests to it) instead of exporting raw key bytes.
- Structure the key manager so it can be swapped for an HSM/OS keystore or separate process without changing call sites.
- Where feasible, use zeroizing wrappers for in-memory secrets to reduce (partial) exposure after use.

Update: *Resolved in pull request #148 at commit e85aed0. The team stated:*

> *- Replaced secret key access in rust clients with a shared signer abstraction.*
>
> *- The base PSM client and the multisig client now both depend on a single Signer interface with* `sign_word`
>
> *- Removed key material getters/cloning*

L-02 Typescript Client Accepts Stale State

In the TypeScript client, the `syncState` function overwrites its local state on any commitment mismatch, without comparing nonces. This means that it is possible to overwrite a newer state with a stale one. For comparison, the Rust client does check the nonces and proceeds with the state update only if the incoming nonce is greater than the locally stored one.

In the TypeScript client, consider comparing the values of the nonces before updating local state.

Update: *Resolved in pull request #148 at commit 3c8df89. The team stated:*

> *- Added a nonce check to the ts multisig before state overwrite.*
>
> *- When `syncState()` sees a commitment mismatch, it only accepts the incoming PSM state if its account nonce is strictly greater than the local nonce, otherwise it rejects the overwrite.*
>
> *- This mirrors the behaviour in the rust client.*

L-03 Unreachable Code

The `sync_account` function is designed to either update the local account if it exists or otherwise retrieve the account from the PSM. However, in the second scenario, the code attempts to read the desired account ID from a non-existent local account `self.require_account()?.id()`, which will fail. Consequently, the `pull_account` call is unreachable, and the branch cannot be executed.

Consider allowing the caller to provide the desired account ID as a parameter, or removing the function entirely so the user must choose whether to call `sync` or `pull_account`.

Update: *Resolved in pull request #148 at commit 472f80b. The team stated:*

> *Removed unused `sync_account()` function*

L-04 Redundant PSM Endpoint Field

The `MultisigClient` and `MultisigAccount` structs both contain (1 and 2) fields for the PSM endpoint.

To avoid confusion or inconsistencies, consider removing one of the fields.

Update: *Resolved in pull request #148 at commit 8a4442b. The team stated:*

> *Removed `psm_endpoint` reference from `MultisigAccount` and kept in `MultisigClient` only*

L-05 TypeScript Client Multisig `load()` Creates Null-Account Instances

When an existing multisig account is loaded from the PSM, a multisig instance is created with `account = null`. The multisig constructor accepts `Account | null` and stores it as-is. `registerOnPsm` uses a non-null assertion `this.account!.serialize()` when `initialStateBase64` is not provided. Thus, if it is called on a multisig created via `load()` without a provided state, it can crash. Such a scenario may occur if the user switches to a new PSM endpoint and wants to register the same account there.

The above is not an issue for the Rust client since `register_on_psm` just calls `push_account` which checks if the account exists and returns an error if it does not. No null account is created in this case.

Consider enforcing that `MultisigClient.load()` passes a real `Account` into `new Multisig`, or guarding `registerOnPsm` (and other methods) to require a non-null account as input.

Update: *Resolved in pull request #148 at commit da1fbf7. The team stated:*

> *- Enforced non-null account invariant in the ts multisig SDK.*
>
> *- `MultisigClient.load()` now always constructs Multisig with an `Account` instance*
>
> *- This removes the null-account state that could crash `registerOnPsm()`*

L-06 Code Simplification

Throughout the codebase, the following opportunities for code simplification were identified:

- After receiving the proposals provided by the PSM, the multisig client infers the transaction type based on the available metadata. This is fragile because it propagates inconsistent metadata and signers that do not match the local account. Instead, the

PSM should specify the transaction type so that the client can retrieve and validate the relevant fields accordingly. A similar observation applies to exported proposals.
- When signing or executing a proposal, the client first downloads all relevant proposals from the PSM and parses them all, only to select the first one that matches the relevant proposal ID. Instead, the client should be able to query the PSM for a single proposal.
- This `else` branch is unnecessary because it reproduces the default behavior below.
- When retrieving cosigner commitments, checking for the first cosigner is redundant because if there are not any, the following loop will exit immediately.
- The `finalize_transaction` function pushes the account to the new server. Since it is known that it does not exist there yet, the more descriptive `register_on_psm` function should be used.
- The `syncProposal` function handles missing metadata before calling `deltaToProposal`, which performs the same operation. The possibly empty metadata could be passed to `deltaToProposal` directly.
- When creating a "remove signer" proposal instead of checking if the signer exists. The code could simply confirm that the filter operation results in a smaller list.

Consider implementing the above-listed code simplifications to improve code clarity and maintainability.

Update: *Resolved in pull request #148 at commit de29ef6 and at commit b612d2f. The team stated:*

> *- Added a single-proposal API to the server contract:*
>
> *- grpc: `GetDeltaProposal(account_id, commitment)`*
>
> *- HTTP: GET `/delta/proposal/single?account_id=...&commitment=...`*
>
> *- Added rust and ts client support: `PsmClient::get_delta_proposal(...)` & `getDeltaProposal(accountId, commitment)`*
>
> *- Refactored rust multisig proposal flows:*
>
> *- `sign_proposal`, `execute_proposal`, and export now fetch one proposal by commitment (no list+scan).*
>
> *- `sign_proposal` uses the sign response delta directly as the updated proposal.*
>
> *- `Proposal::from(...)` now parses transaction type strictly from `metadata.proposal_type` (removed inference logic).*

> *- Added explicit proposal signature entries on Proposal and used them in export / import.*
>
> *- Removed redundant transaction type helpe*
>
> *- In finalize / switch-PSM path, changed from `push_account()` to `register_on_psm()`.*
>
> *- Refactored typescript multisig proposal flows:*
>
> *- `executeProposal` and `exportProposal` now call `getDeltaProposal(...)`*
>
> *- Tightened metadata parsing / validation (mandatory proposal type + required fields)*
>
> *- Tightened import validation (nonce required, metadata / proposalType required)*
>
> *- Simplified remove-signer creation check via filtered-list size comparison.*
>
> *- Server payload normalization/validation*
>
> *- Enforced valid `tx_summary`*
>
> *- Enforced required `metadata.proposal_type` and normalized metadata fields like `required_signatures` / `amount`.*

L-07 Documentation Suggestions

Throughout the codebase, multiple opportunities for improving the documentation were identified:

- The comment in `finalize_transaction` states that the error is to be logged. However, no logging is performed (empty block).
- In the Quick Start example in the Miden Multisig Client SDK lib file, consider adding a call to `config.validate()` after creating a new instance with `MultisigConfig::new` in order to adhere to best practices.

Consider implementing the above-listed suggestions to improve the clarity and readability of the codebase.

Update: *Resolved in pull request #148 at commit 33696ac. The team stated:*

> *Fixed documentation points*

L-08 Users Can Sign Unrelated Proposal

When signing a proposal, the multisig client confirms that the key manager matches the local account. However, it is not compared to the `account_id` of the actual proposal under consideration. This means that the `sign_proposal` function can be used to sign a proposal for an unrelated account that happens to share the same PSM. A similar observation applies to imported proposals.

Note that in the online case, the `sign_proposal` function operates on proposals fetched from `list_proposals`. The latter gets the `account` and `account_id` of the client account and then queries the PSM server on that `account_id`. Therefore, an honest PSM server will normally only return proposals matching the signing account ID. Although, this may not be the case for a malicious or buggy PSM server.

Consider checking the account ID before signing a proposal.

Update: *Resolved in pull request #148 at commit c4adbf1. The team stated:*

> *- implemented `ensure_proposal_account_id` (rust client) and `assertProposalAccountId` (typescript client)*
>
> *- run assertions every time a proposal is fetched from the server or imported for signing.*

L-09 Wraparound Risk for Untrusted Inputs in Hex-to-Word Parsing

The multisig PSM client parses hex strings into Word with `hex_to_word` by splitting into 64-bit limbs and calling `Felt::new`, which reduces modulo the field. This silently changes values when limbs are `>=` field modulus.

The described approach is safe only when the hex originated from a canonical Word serialization and becomes risky for untrusted inputs such as offline imports, PSM-provided metadata, tampered JSON, etc. For instance, `ProposalMetadata::salt()` uses `hex_to_word` on `salt_hex` from metadata (untrusted in offline/PSM parsing). Similarly, `SwitchPsm` parsing converts `new_psm_pubkey_hex` with `hex_to_word` (untrusted metadata).

Consider rejecting non-canonical hex by validating that each 64-bit limb is less than the field modulus before converting to `Felt`.

Update: *Resolved in pull request #148 at commit 30106c0. The team stated:*

> *- add assertions to `word_from_hex` to reject non-canonical hex values with invalid field elements*
>
> *- Use `word_from_hex` consistently across codebase, avoiding code repetition*
>
> *- remove repeated implementations*

Notes & Additional Information

N-01 Ambiguous Signer Map Selection When Both OZ and Miden Slots Are Present

In the Rust client, the `cosigner_commitments` function returns the commitments of the multisig account cosigners. The valid cosigners are retrieved from the signer map, for either `OZ_MULTISIG_SIGNER_PUBKEYS` (OpenZeppelin slot name) or `STD_APPROVER_PUBKEYS` (Miden slot name). This may be ambiguous if both slots are non-empty, ultimately returning unexpected results.

Consider using a single slot name. Alternatively, consider adding an explicit precedence order so that the secondary slot will only be checked if the primary slot is empty.

Update: *Resolved in pull request #148 at commit 21697a1. The team stated:*

> *- removed deprecated references to miden multisig templates slots*
>
> *- simplified the slot inspection logic, avoiding unnecesary fallbacks*

N-02 Incomplete Proposal Description

The `createP2idProposal` function describes the proposal as a transfer of tokens to a recipient without mentioning the actual token asset.

Consider including the asset in the description.

Update: *Resolved in pull request #148 at commit bb853f3. The team stated:*

> *Added asset id to p2id proposal description*

N-03 Inconsistent Codebase

The Typescript multisig client defines constants to describe the Miden contract libraries. These are used when creating new accounts. However, these constants do not match the actual contracts in the codebase.

To avoid confusion, consider ensuring that both descriptions correspond to the latest contract code. Alternatively, consider defining the Typescript constants by reading the files.

Update: *Resolved in pull request #148 at commit f7b5791. The team stated:*

> *- fixed inconsistencies across masm templates*
>
> *- auto-generates the typescript masm code (as constants) in build-time using the masm templates (matching the rust implementation)*

N-04 Unused Code

Throughout the codebase, multiple instances of unused code were identified:

- The `MultiSigConfig` struct.
- The `_account_id` variable.

Consider removing unused code to improve the clarity and maintainability of the codebase.

Update: *Resolved in pull request #148 at commit 90fbb91. The team stated:*

> *- removed config.rs which carried mostly dead code (including `MultisigConfig`), and moved used code to other files*
>
> *- remove unused _account_id var*

Client Reported

CR-01 PSM Acknowledgement Never Expires

Before executing a proposal, the multisig client retrieves an acknowledgement (ack) from the PSM to ensure that the PSM knows about the latest state. The PSM will consider this new transaction as a "candidate" update. However, if the transaction is not posted to the blockchain within a short window, the PSM will discard the candidate, reverting to the previous confirmed state. The client could then post the transaction, updating the state of the account, but the PSM will remain permanently out of date.

Consider ensuring that the PSM acknowledgement expires at a particular block number so the PSM knows that the candidate transaction is safe to discard.

Update: *Acknowledged, will resolve. The team stated:*

> *An approach suggested by Bobbin (CTO at Miden) is to use transaction expirations, Miden supports expiration for transactions, which we could leverage for solving this issue.*
>
> *- By default all the transactions in the multisig should have an expiration*
>
> *- PSM will check the expiration is present/valid when the user tries to push a new transaction.*
>
> *- If the expiration is set to some valid block window, then the PSM will sign over it, otherwise rejects.*
>
> *- when canonicalizing (monitoring the on-chain settlement) will discard / confirm the the transaction only after the defined block window expired*
>
> *- if an attacker tries to hold the psm ack until the canonicalization discards the tx. The tx will fail on-chain because it's expired*

Conclusion

This audit reviewed the multisig PSM client implementations in Rust (`crates/miden-multisig-client`) and TypeScript (`packages/miden-multisig-client`). The scope covered account handling, proposal creation and signing, proposal execution, PSM coordination, offline import/export, and related utilities used to build and verify transactions.

The most notable findings involve potential inconsistencies between the cryptographically protected values and the associated contextual information. The secondary findings mainly pertain to robustness and integrity issues. These include differences between Rust and TypeScript validation paths, reliance on untrusted PSM metadata with defaulted fields, a TS state-sync path that can accept stale snapshots, and missing defensive checks (e.g., duplicate signer commitments at account creation). It is recommended to put a strong focus on simplification and defensive programming as it could improve the general resilience of the codebase.

Overall, the codebase was found to be structured and reasonably consistent across languages, with clear separation between proposal building, execution, and account management. The Rust implementation is generally more defensive and explicit about validation steps, while the TypeScript implementation is more permissive and therefore more sensitive to malformed PSM data. With a small set of targeted hardening changes, the clients can better align on safety guarantees and reduce operational failure modes.

The final state of the audited codebase, including all implemented resolutions, is reflected in commit eb89f8d.

The Miden development team is commended for their responsiveness throughout the audit. Special thanks are extended to Marcos for his prompt and detailed answers to all our technical questions.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.